

Deployment Guide

Enterprise Supply Chain Demand Forecasting & Inventory Optimization Platform

Prepared by: Muhammad Hassan Tariq

Table of Contents

Table of Contents	2
1. Introduction	7
1.1 Project Overview	7
1.2 Deployment Objectives	7
1.3 System Architecture Overview	7
2. System Requirements	7
2.1 Hardware Requirements	7
2.2 Operating System Requirements	7
2.3 Python Requirements	7
2.4 Database Requirements	8
2.5 Redis Requirements	8
3. Technology Stack	8
3.1 Backend: FastAPI	8
3.2 Database: PostgreSQL	8
3.3 Cache: Redis	8
3.4 Machine Learning Stack	8
3.5 MLOps: MLflow	8
3.6 Dashboard: Streamlit	9
4. Project Structure	9
4.1 Directory Structure	9
4.2 Configuration Files	9
4.3 Environment Variables	9
5. Environment Setup	10
5.1 Python Virtual Environment	10
5.2 Installing Dependencies	10
5.3 Environment Configuration	10
5.4 PostgreSQL Setup	10
5.5 Redis Setup	10

5.6 MLflow Setup	10
6. Database Deployment	10
6.1 PostgreSQL Database Creation	10
6.2 Database Schema	10
6.3 Data Loading	10
6.4 Database Verification	11
7. ML Pipeline Deployment	11
7.1 Data Pipeline	11
7.2 Feature Engineering Pipeline	11
7.3 Model Training	11
7.4 Model Evaluation	11
7.5 Model Registration	11
7.6 Model Loading for Production	11
8. MLOps Deployment	11
8.1 MLflow Configuration	11
8.2 Experiment Tracking	12
8.3 Model Registry	12
8.4 Data Drift Monitoring	12
8.5 Automated Retraining	12
8.6 Model Performance Monitoring	12
9. FastAPI Backend Deployment	12
9.1 API Configuration	12
9.2 Starting the FastAPI Server	12
9.3 API Authentication	12
9.4 REST API Endpoints	13
9.5 API Documentation	13
9.6 Health Checks	13
10. Redis Configuration	13
10.1 Redis Connection	13

10.2 Dashboard Caching	13
10.3 Alert Caching	13
10.4 Cache Verification	13
11. Streamlit Dashboard Deployment	14
11.1 Dashboard Configuration	14
11.2 Starting the Dashboard	14
11.3 API Integration	14
11.4 Dashboard Verification	14
12. Production Configuration	14
12.1 Environment Variables	14
12.2 Security Configuration	14
12.3 Database Security	14
12.4 API Security	15
12.5 Logging	15
13. Scalability & High Availability	15
13.1 Stateless API Architecture	15
13.2 Horizontal Scaling Strategy	15
13.3 Redis Caching Strategy	15
13.4 Database Scaling Considerations	15
13.5 Load Balancer Architecture	15
13.6 100 Million Forecast Request Scalability Strategy	15
14. Monitoring & Maintenance	16
14.1 API Monitoring	16
14.2 Model Monitoring	16
14.3 Data Drift Monitoring	16
14.4 Forecast Drift	16
15. Backup & Recovery	16
15.1 Database Backup	16
15.2 Model Backup	16

16. Testing & Validation	17
16.1 API Testing	17
16.2 Database Testing	17
16.3 Forecasting Validation	18
16.4 Inventory Optimization Validation	18
Low Inventory Scenario	18
High Inventory Scenario	19
High Demand Scenario	19
Warehouse Transfer Scenario	19
16.5 Dashboard Validation	19
17. Troubleshooting	19
17.1 FastAPI Issues	19
Server does not start	19
API returns 401 Unauthorized	19
Model is not loaded	19
17.2 PostgreSQL Issues	20
Database connection fails	20
Table does not exist	20
Duplicate records appear	20
17.3 Redis Issues	20
Redis connection fails	20
Dashboard data appears stale	20
17.4 MLflow Issues	21
MLflow database cannot be accessed	21
Registered model cannot be found	21
18. Conclusion	21
19. Important Commands	21
Create Virtual Environment	21
Activate Virtual Environment for Windows	21

Install Dependencies	21
Generate Data	21
Run Feature Engineering	22
Train Models	22
Create Database Schema	22
Load Data into PostgreSQL	22
Run Drift Monitoring	22
Run Model Monitoring	22
Start FastAPI	22
Start Streamlit	22
Test Redis	22

1. Introduction

1.1 Project Overview

The ML-021 Enterprise Supply Chain Demand Forecasting & Inventory Optimization Platform is an AI-powered system designed to forecast product demand, optimize inventory levels, identify supply chain risks, and support procurement planning.

The platform combines machine learning, time-series forecasting, inventory optimization, explainable AI, risk detection, scenario simulation, MLOps, REST APIs, PostgreSQL, Redis, and an executive dashboard.

1.2 Deployment Objectives

- Deploy the forecasting and inventory optimization services.
- Provide secure REST APIs through FastAPI.
- Connect the application to PostgreSQL.
- Use Redis for high-speed caching.
- Deploy and monitor machine learning models.
- Provide access to the Streamlit executive dashboard.
- Support production-oriented monitoring and retraining.

1.3 System Architecture Overview

The platform follows a modular architecture consisting of data processing, feature engineering, machine learning, inventory optimization, risk detection, MLOps, backend APIs, database services, and dashboard components.

2. System Requirements

2.1 Hardware Requirements

- CPU: 2 or more cores recommended.
- RAM: 8 GB minimum; 16 GB recommended.
- Storage: 10 GB or more available space.
- Stable network connection for package installation and deployment.

2.2 Operating System Requirements

- Windows 10/11, Linux, or macOS.
- 64-bit operating system recommended.

2.3 Python Requirements

- Python 3.11 (matches the `venv` used throughout development).

- **Key libraries:** pandas, numpy, scikit-learn, xgboost, lightgbm, prophet, mlflow, shap, optuna, fastapi, uvicorn, sqlalchemy, psycopg2-binary, redis, streamlit, python-dotenv, faker.

2.4 Database Requirements

- PostgreSQL 18 recommended.

2.5 Redis Requirements

- Redis 6+ recommended.

3. Technology Stack

3.1 Backend: FastAPI

FastAPI provides REST APIs for forecasting, inventory recommendations, transfers, alerts, and dashboard data.

3.2 Database: PostgreSQL

PostgreSQL stores application data, product information, inventory, forecast results, recommendations, alerts, and related records.

3.3 Cache: Redis

Redis is used to cache frequently requested dashboard and alert data to reduce response latency.

3.4 Machine Learning Stack

- Pandas
- NumPy
- Scikit-learn
- XGBoost
- LightGBM
- Prophet
- SHAP
- Optuna

3.5 MLOps: MLflow

- Experiment tracking
- Model registry
- Model versioning
- Model monitoring

3.6 Dashboard: Streamlit

Streamlit provides the executive interface for supply chain KPIs, forecast performance, inventory health, warehouse utilization, stockout probability, procurement recommendations, and alerts.

4. Project Structure

4.1 Directory Structure

Project folder/

```
├── data/{raw,processed}/
├── src/
│   ├── data_generation.py
│   ├── feature_engineering.py
│   ├── forecasting/ (train.py, hyperparameter_tuning.py, feature_selection.py,
│   │               multi_granularity.py, explainability.py)
│   ├── inventory/ (optimization.py, allocation.py)
│   ├── risk/ (detection.py, capacity_and_drift.py)
│   ├── simulation/ (scenario_engine.py)
│   ├── mlops/ (drift_monitoring.py, model_monitoring.py, export_metrics.py)
│   ├── db/ (schema.py, load_to_postgres.py)
│   └── api/ (main.py)
├── dashboard/app.py
├── reports/
├── mlflow/
└── requirements.txt
```

4.2 Configuration Files

- requirements.txt
- .env

4.3 Environment Variables

Variable	Example
POSTGRES_URL	postgresql://postgres:your_password@localhost:5432/db_name
REDIS_URL	redis://localhost:6379/0
FORECAST_API_KEY	dev-secret-key-change-in-production

5. Environment Setup

5.1 Python Virtual Environment

```
python -m venv venv
```

```
# Windows  
venv\Scripts\activate
```

5.2 Installing Dependencies

- `pip install -r requirements.txt`
- `pip install psycpg2-binary sqlalchemy redis python-dotenv optuna`

5.3 Environment Configuration

Create `.env` in the project root with the three variables listed above.

5.4 PostgreSQL Setup

- i. Install PostgreSQL for Windows, set the **postgres** superuser password during setup.
- ii. Create the database:

```
psql -U postgres -c "CREATE DATABASE your_DB_name;"
```

5.5 Redis Setup

Install Memurai (Redis-compatible Windows service). No further configuration needed, it listens on the default port 6379.

5.6 MLflow Setup

No separate install step, MLflow is a pip dependency and uses a local SQLite file (**mlflow/mlflow.db**) as its backing store, created automatically on first run.

6. Database Deployment

6.1 PostgreSQL Database Creation

Create the PostgreSQL database and configure the application's connection string.

6.2 Database Schema

```
python src/db/schema.py
```

6.3 Data Loading

```
python src/db/load_to_postgres.py
```

Loads all processed pipeline outputs (inventory plan, alerts, transfers, allocation, warehouse utilization, daily forecasts, latest model metrics) into PostgreSQL. **Note:** this appends, re-running without truncating first will duplicate rows. Truncate before a full re-load:

```
psql -U postgres -d your_db_name -c "TRUNCATE inventory_plan, alerts, transfer_recommendations, warehouse_allocation, warehouse_utilization, daily_forecast, model_metrics;"
```

6.4 Database Verification

```
psql -U postgres -d your_db_name -c "SELECT COUNT(*) FROM inventory_plan;"
```

7. ML Pipeline Deployment

7.1 Data Pipeline

```
python src/data_generation.py
```

7.2 Feature Engineering Pipeline

```
python src/feature_engineering.py
```

7.3 Model Training

```
python src/forecasting/train.py
```

7.4 Model Evaluation

See the separate Model Evaluation Report for full metrics and analysis.

7.5 Model Registration

Handled automatically by **train.py** via **mlflow.xgboost.log_model(..., registered_model_name=...)**.

Each re-run creates a new version under the same registered model name.

7.6 Model Loading for Production

FastAPI loads the latest registered version at startup:

```
mlflow.pyfunc.load_model("models:/demand_forecast_xgboost/latest")
```

8. MLOps Deployment

8.1 MLflow Configuration

MLflow tracks experiments, parameters, metrics, artifacts, and model versions.

8.2 Experiment Tracking

- Track model parameters.
- Track validation metrics.
- Store trained model artifacts.
- Compare experiments.

8.3 Model Registry

demand_forecast_xgboost, **demand_forecast_lightgbm**, **demand_forecast_xgboost_tuned**, versioned automatically on each training run.

8.4 Data Drift Monitoring

```
python src/mlops/drift_monitoring.py
```

8.5 Automated Retraining

If more than 15% of features are flagged as drifted, **drift_monitoring.py** invokes **train.py** as a subprocess, a genuine pipeline trigger, not a logged message. Verified in testing: 24.14% of features drifted, which triggered a real retraining run that registered model version 2.

8.6 Model Performance Monitoring

```
python src/mlops/model_monitoring.py
```

Distinct from drift monitoring, compares the model's actual recent prediction error (last 14 days, MAE/MAPE against logged predictions) to its baseline training MAE, flagging degradation beyond a 1.15x threshold.

9. FastAPI Backend Deployment

9.1 API Configuration

Reads **POSTGRES_URL**, **REDIS_URL**, and **FORECAST_API_KEY** from **.env** at import time via **python-dotenv**.

9.2 Starting the FastAPI Server

```
uvicorn src.api.main:app --reload --port 8000
```

9.3 API Authentication

All endpoints except **/health** require an **X-API-Key** header matching **FORECAST_API_KEY**; requests without it receive **401 Unauthorized**.

9.4 REST API Endpoints

Endpoint	Description
/health	Liveness + model-loaded check
/forecast/predict	Single-row demand prediction
/inventory/recommendations	Reorder point, safety stock, EOQ, stockout probability
/inventory/transfers	Inter-warehouse transfer suggestions
/inventory/allocation	New-procurement allocation by warehouse
/alerts	Consolidated, severity-ranked alert feed
/dashboard/summary	All executive KPIs in one call

9.5 API Documentation

Swagger UI: <http://127.0.0.1:8000/docs> (interactive, includes an "Authorize" button for the API key).

9.6 Health Checks

GET /health returns `{"status": "ok", "model_loaded": true}` once the MLflow model has loaded successfully at startup.

10. Redis Configuration

10.1 Redis Connection

`redis.from_url(REDIS_URL, decode_responses=True)`, a single shared client instantiated at FastAPI startup.

10.2 Dashboard Caching

`/dashboard/summary` checks Redis first; on a miss it runs the PostgreSQL aggregation queries and caches the JSON result for 30 seconds.

10.3 Alert Caching

`/alerts` uses a parameterized cache key (`alerts:{severity}:{alert_type}:{limit}`) so different filter combinations cache independently.

10.4 Cache Verification

```
python -c "import redis; r = redis.Redis(); print(r.ping())"
```

Expected: True

11. Streamlit Dashboard Deployment

11.1 Dashboard Configuration

`API_BASE = "http://127.0.0.1:8000"` and the shared `FORECAST_API_KEY` are set at the top of `dashboard/app.py`.

11.2 Starting the Dashboard

```
streamlit run dashboard/app.py
```

Requires the FastAPI server to already be running in a separate terminal.

11.3 API Integration

A single `fetch()` helper wraps `requests.get()` with the API key header and 30-second `st.cache_data` caching, used by every dashboard section.

11.4 Dashboard Verification

- Forecast Accuracy
- Inventory Health
- Warehouse Utilization
- Fill Rate
- Stockout Probability
- Procurement Recommendations
- Supply Chain KPIs

12. Production Configuration

12.1 Environment Variables

Never hard-code credentials, all three secrets load from `.env` (excluded from version control).

12.2 Security Configuration

API-key header auth is the current control.

12.3 Database Security

Development uses the postgres superuser for simplicity. Production should use a dedicated least-privilege role scoped to the 7 application tables.

12.4 API Security

Consider rate limiting and rotating the API key for production (not implemented in this case-study deployment).

12.5 Logging

Uvicorn's default access/error logs are used; no structured/centralized logging was set up (noted as future work).

13. Scalability & High Availability

13.1 Stateless API Architecture

FastAPI holds no per-request session state, the loaded MLflow model is read-only, and all mutable state lives in PostgreSQL/Redis. This is what makes horizontal replication safe.

13.2 Horizontal Scaling Strategy

The API can be deployed as multiple independent instances behind a load balancer. PostgreSQL remains the persistent data layer while Redis provides shared caching.

13.3 Redis Caching Strategy

30-second TTL caching on the two highest-traffic, aggregation-heavy endpoints (`/dashboard/summary`, `/alerts`) absorbs repeated dashboard polling without hitting PostgreSQL on every request.

13.4 Database Scaling Considerations

For production scale, read replicas and connection pooling (e.g. PgBouncer) would sit in front of PostgreSQL; out of scope for this deployment.

13.5 Load Balancer Architecture

In production, a load balancer such as Nginx or a cloud load-balancing service can distribute API requests across multiple FastAPI instances.

13.6 100 Million Forecast Request Scalability Strategy

This project was **not tested with a large number of users** because it was developed by one person on a single computer. Testing at that scale was not practical.

However, the system is **designed to handle more users in the future** by:

- Running multiple FastAPI servers.
- Using a load balancer to distribute requests between servers.
- Using Redis to quickly return frequently requested data.
- Using PostgreSQL read replicas to handle many users reading data at the same time.

14. Monitoring & Maintenance

14.1 API Monitoring

`/health` endpoint suitable for uptime checks; no external monitoring tool was wired up in this deployment.

14.2 Model Monitoring

`src/mlops/model_monitoring.py`

14.3 Data Drift Monitoring

`src/mlops/drift_monitoring.py`

14.4 Forecast Drift

Surfaced as a **FORECAST_DRIFT** alert (HIGH severity) in the Alert Center whenever the drifted-feature ratio exceeds 15%, via `src/risk/capacity_and_drift.py`.

15. Backup & Recovery

15.1 Database Backup

```
pg_dump -U postgres ml021_supply_chain > backup.sql
```

15.2 Model Backup

MLflow artifacts (model binaries) live under the MLflow tracking store; back up the **mlflow/** directory alongside **mlflow.db**.

16. Testing & Validation

16.1 API Testing

The FastAPI backend was tested to verify that the implemented REST endpoints respond correctly and enforce API authentication.

The following areas were validated:

- /health returns the application and model loading status.
- /forecast/predict accepts forecasting input and returns a demand prediction.
- /inventory/recommendations returns inventory optimization recommendations.
- /inventory/transfers returns inter-warehouse transfer suggestions.
- /inventory/allocation returns procurement allocation recommendations.
- /alerts returns severity-ranked supply chain alerts.
- /dashboard/summary returns the executive dashboard KPIs.
- Protected endpoints reject requests without a valid X-API-Key.
- Invalid API keys return 401 Unauthorized.
- The /health endpoint remains accessible without authentication.

Interactive API testing can be performed through the FastAPI Swagger interface at:

<http://127.0.0.1:8000/docs>

16.2 Database Testing

PostgreSQL connectivity and application tables were verified after database initialization and data loading.

Database validation includes:

- Confirming that the ml021_supply_chain database exists.
- Verifying that all required application tables are created.
- Checking row counts after data loading.
- Verifying that forecast records are available.
- Verifying inventory optimization results.
- Verifying transfer and allocation recommendations.
- Verifying alert records.
- Confirming that FastAPI can successfully query PostgreSQL.

Example verification command:

```
psql -U postgres -d ml021_supply_chain -c "SELECT COUNT(*) FROM inventory_plan;"
```

Additional table-level checks should be performed when a complete pipeline reload is executed.

Important: The current loading script appends records. A full reload should therefore be preceded by an appropriate truncate/cleanup operation to prevent duplicate records.

16.3 Forecasting Validation

The demand forecasting pipeline was validated using time-series-aware validation rather than random train/test splitting.

The validation process includes:

1. Preparing historical demand data.
2. Creating time-dependent features.
3. Maintaining chronological ordering.
4. Training forecasting models on historical observations.
5. Evaluating predictions on later observations.
6. Comparing model performance using forecasting metrics.
7. Selecting the best-performing model for production use.

The primary evaluation metrics include:

- MAE: Mean Absolute Error
- RMSE: Root Mean Squared Error
- MAPE: Mean Absolute Percentage Error

Time-series cross-validation helps prevent future information from being unintentionally introduced into the training data.

The complete model metrics and experiment results are documented separately in the Model Evaluation Report.

16.4 Inventory Optimization Validation

The inventory optimization engine was validated to ensure that calculated recommendations are logically consistent with demand and inventory conditions.

The following components were checked:

- Reorder Point
- Safety Stock
- Economic Order Quantity (EOQ)
- Stockout Probability
- Procurement Quantity
- Warehouse Allocation
- Inter-Warehouse Transfers

Validation scenarios include:

Low Inventory Scenario

When projected inventory falls below the calculated reorder threshold, the system should generate an appropriate replenishment recommendation.

High Inventory Scenario

When inventory substantially exceeds expected demand, the system should identify potential overstock conditions.

High Demand Scenario

When forecasted demand increases significantly, the system should increase the calculated replenishment requirement where appropriate.

Warehouse Transfer Scenario

When one warehouse has excess inventory while another warehouse has insufficient inventory, the transfer engine can generate an inter-warehouse transfer recommendation.

These tests verify that inventory recommendations are connected to forecast demand and operational constraints rather than being static values.

16.5 Dashboard Validation

Dashboard validation also confirms that:

- API requests include the required API key.
- Data is retrieved successfully from FastAPI.
- Dashboard sections display the returned data correctly.
- Cached responses are refreshed according to the configured cache duration.
- API failures do not silently display incorrect data.

17. Troubleshooting

17.1 FastAPI Issues

Server does not start

Verify that the virtual environment is active and dependencies are installed.

```
venv\Scripts\activate  
pip install -r requirements.txt
```

Start the server again:

```
uvicorn src.api.main:app --reload --port 8000
```

API returns 401 Unauthorized

Verify that the request contains:

X-API-Key: <your configured API key>

Also confirm that FORECAST_API_KEY is correctly configured in .env.

Model is not loaded

Check that the MLflow tracking store is available and that the registered model exists.

The /health endpoint should indicate whether the model has loaded successfully.

17.2 PostgreSQL Issues

Database connection fails

Check:

- PostgreSQL service is running.
- Database name is correct.
- Username and password are correct.
- POSTGRES_URL is correctly configured.
- PostgreSQL is listening on the expected port.

Example:

```
postgres://postgres:your_password@localhost:5432/your_db_name
```

Table does not exist

Run the database schema script:

```
python src/db/schema.py
```

Then run the data-loading process:

```
python src/db/load_to_postgres.py
```

Duplicate records appear

The current data-loading process appends records. If the loader is executed multiple times, duplicate rows can be created.

Before a complete reload, clear the relevant application tables according to the project's database procedure.

17.3 Redis Issues

Redis connection fails

Verify that the Redis-compatible service is running and listening on port 6379.

Test the connection:

```
python -c "import redis; r = redis.Redis(); print(r.ping())"
```

Expected result:

```
True
```

Dashboard data appears stale

The dashboard summary and alert endpoints use caching.

The current API cache uses a 30-second TTL. Waiting for the TTL to expire or restarting the Redis service can be used during development testing.

17.4 MLflow Issues

MLflow database cannot be accessed

Verify that the MLflow tracking directory exists and that the application has permission to access it.

The local MLflow tracking database is stored under:

```
mlflow/mlflow.db
```

Registered model cannot be found

Run the model training pipeline again:

```
python src/forecasting/train.py
```

Then verify that the expected registered model is available.

The production API expects the configured registered model to be available through MLflow.

18. Conclusion

The Enterprise Supply Chain Demand Forecasting & Inventory Optimization Platform provides an integrated machine learning solution for **demand forecasting, inventory optimization, supply-chain risk detection, and procurement planning**.

The platform combines **time-series feature engineering, machine learning models, inventory optimization, explainable AI, MLOps, FastAPI, PostgreSQL, Redis, and a Streamlit executive dashboard** into a modular end-to-end system.

The project was successfully validated in a single development environment. Although enterprise-scale performance, including **100 million forecast requests**, was not directly load-tested, the architecture supports future scaling through **stateless APIs, shared caching, database scaling, and horizontal deployment**.

19. Important Commands

Create Virtual Environment

```
python -m venv venv
```

Activate Virtual Environment for Windows

```
venv\Scripts\activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Generate Data

```
python src/data_generation.py
```

Run Feature Engineering

```
python src/feature_engineering.py
```

Train Models

```
python src/forecasting/train.py
```

Create Database Schema

```
python src/db/schema.py
```

Load Data into PostgreSQL

```
python src/db/load_to_postgres.py
```

Run Drift Monitoring

```
python src/mlops/drift_monitoring.py
```

Run Model Monitoring

```
python src/mlops/model_monitoring.py
```

Start FastAPI

```
uvicorn src.api.main:app --reload --port 8000
```

Start Streamlit

```
streamlit run dashboard/app.py
```

Test Redis

```
python -c "import redis; r = redis.Redis(); print(r.ping())"
```